“UNIVERSIDAD MICHOACANA SAN NICOLÁS DE HIDALGO”

MODELOS PROBABILISTAS

PROYECTO INTEGRADOR (Parte 1): DESARROLLO DE UNA APLICACIÓN GRÁFICA PARA EL MODELADO Y LA RESOLUCIÓN DE REDES BAYESIANAS

ALUMNOS: **MOISES MORENO CORTEZ**
LEONARDO SORIA RAYA
CESAR AMBRIS SALGUERO

MAESTRO: M.S.C. MAURICIO RENE REYES GUTIERREZ

FECHA DE ENTREGA 03/11/2026

ÍNDICE

Planteamiento del Problema 4

Objetivos 5

Fundamentos Teóricos: Redes Bayesianas y Eliminación de Variables 6

Justificación de las Herramientas Seleccionadas 10

Arquitectura del Software y Diseño de Componentes 11

epresentación Interna de Nodos, Relaciones y Distribuciones 13

Explicación Detallada del Algoritmo Implementado 14

Pruebas, Resultados, Limitaciones y Posibles Mejoras 18

Repositorio Oficial de GitHub 20

Conclusiones 21

Distribución de Responsabilidades y Contribuciones Individuales 22

INTRODUCCIÓN

El razonamiento bajo incertidumbre constituye uno de los desafíos medulares en las ciencias de la computación y la inteligencia artificial. En escenarios del mundo real, la información suele presentarse incompleta, con ruido o sujeta a dependencias complejas. Los Modelos Gráficos Probabilistas (PGM, por sus siglas en inglés), y de forma paradigmática las **Redes Bayesianas**, proporcionan un marco formal, riguroso y compacto para representar distribuciones de probabilidad conjuntas de alta dimensión mediante la factorización modular inducida por independencias condicionales

El presente proyecto documenta la especificación, diseño, arquitectura, implementación y validación de una aplicación de escritorio multiplataforma con Interfaz Gráfica de Usuario (GUI). El sistema permite construir interactivamente grafos dirigidos acíclicos (DAGs), definir y parametrizar distribuciones de probabilidad condicional (CPDs) discretas, validar la consistencia estructural y estocástica de los modelos, y ejecutar consultas probabilistas exactas mediante el algoritmo de **Eliminación de Variables** (*Variable Elimination*), con soporte para evidencia observada arbitraria

PLANTEAMIENTO DEL PROBLEMA

La inferencia probabilista exacta en modelos bayesianos arbitrarios es un problema computacionalmente intratable, clasificado formalmente como #P-difícil. Si se evalúa una distribución de probabilidad conjunta mediante la suma exhaustiva (fuerza bruta), la complejidad espacial y temporal crece exponencialmente con respecto a la cantidad de variables involucradas:

$$\mathcal{O}(k^n)$$

donde n es el número de variables aleatorias y k la cardinalidad máxima de sus estados discretos.

En los entornos pedagógicos y de prototipado rápido, los estudiantes y desarrolladores se enfrentan frecuentemente a dos extremos indeseables:

1. **Librerías especializadas basadas exclusivamente en consola / APIs** (como pgmpy o libpgm), las cuales carecen de retroalimentación visual directa para validar la topología del grafo o inspeccionar paso a paso la transformación de factores.
2. **Plataformas propietarias o anticuadas** (como GeNIe/SMILE o Netica), que presentan restricciones de licencia, dependencias cerradas o nula extensibilidad para fines didácticos en entornos multiplataforma (Linux, macOS, Windows).

Por tanto, se requiere un software robusto, de código abierto, interactivo y multiplataforma que combine la facilidad de edición gráfica (arrastrar y soltar nodos, conectar arcos dirigidos, editar tablas dinámicas de probabilidad) con una biblioteca subyacente matemáticamente transparente para la manipulación y eliminación de factores discretos.

OBJETIVOS

Objetivo General

Diseñar, implementar y evaluar un sistema de software con interfaz gráfica multiplataforma para la modelación interactiva, validación formal y ejecución de inferencia probabilista exacta mediante el algoritmo de Eliminación de Variables en Redes Bayesianas discretas

Objetivos Específicos

1. **Modelado Topológico y Validación Estructural:** Proveer un lienzo gráfico interactivo para la creación, edición y supresión de nodos y arcos dirigidos, garantizando la preservación estricta de la propiedad de aciclicidad (detección de ciclos por ordenamiento topológico o búsqueda en profundidad).
2. **Definición y Verificación Estocástica:** Implementar un editor tabular dinámico para las Tablas de Probabilidad Condicional (CPT), que valide la normalización de las distribuciones:

$$\sum_{x \in \mathrm{Val}(X)} P\left(X = x \mid \mathrm{Padres}(X)\right) = 1.0 \pm \epsilon$$

3. **Motor de Inferencia Exacta:** Construir una librería de álgebra de factores probabilistas (operaciones de producto tensorial, marginación/suma e instanciación de evidencia) y aplicar el algoritmo de Eliminación de Variables optimizado mediante heurísticas de ordenamiento (ej. Min-Neighbors y Min-Weight).

4. **Inspección y Trazabilidad:** Proporcionar al usuario no solo la distribución marginal resultante a posteriori, sino también la traza de factores intermedios generados a lo largo del proceso de inferencia.
5. **Persistencia y Compatibilidad:** Permitir la serialización y deserialización de modelos en un formato de intercambio estándar y legible (JSON / BIF).

FUNDAMENTOS TEÓRICOS: REDES BAYESIANAS Y ELIMINACIÓN DE VARIABLES

Formalización de Redes Bayesianas

Una Red Bayesiana es un par $\mathcal{B} = (\mathcal{G}, \mathcal{P})$, donde:

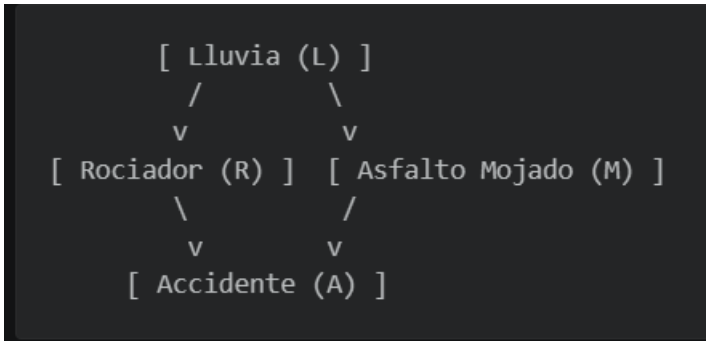
1. $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ es un Grafo Dirigido Acíclico (DAG) cuyos vértices $\mathcal{V} = \{X_1, X_2, \dots, X_n\}$ corresponden a variables aleatorias y cuyas aristas dirigidas $(X_i, X_j) \in \mathcal{E}$ representan dependencias condicionales directas (donde X_i es padre directo de X_j).
2. $\mathcal{P} = \left\{P\left(X_i \mid \text{Padres}(X_i)\right): i \in \{1, \dots, n\}\right\}$ es un conjunto de distribuciones de probabilidad condicional (CPDs) locales asociadas a cada nodo.

Factorización de la Distribución Conjunta

En virtud de la regla de la cadena probabilista y la propiedad de Markov local (cada variable es condicionalmente independiente de sus no-descendientes dados sus padres), la probabilidad conjunta completa sobre todas las variables se factoriza de forma compacta como:

$$P(X_1, X_2, \dots, X_n) = \prod_{i=1}^{n} P\left(X_i \mid \text{Padres}(X_i)\right)$$

Esta descomposición reduce dramáticamente el número de parámetros requeridos para especificar el modelo, pasando de una complejidad exponencial en el peor caso a un crecimiento gobernado por el grado de entrada (*in-degree*) máximo de los nodos.



Concepto de Factor

Un factor $\phi(Y)$ es una función que mapea cada instanciación de un conjunto de variables aleatorias $Y \subseteq \mathcal{V}$ a un número real no negativo:

$$\phi: \mathrm{Val}(Y) \to \mathbb{R}_{\geq 0}$$

A diferencia de una distribución de probabilidad estricta, un factor no requiere sumar 1 sobre ningún dominio particular hasta que es explícitamente normalizado.

Operación de Reducción (Instanciación de Evidencia)

Sea $\phi(Y)$ un factor y $E = e$ una evidencia observada, donde $E \in Y$ y $e \in \mathrm{Val}(E)$. El factor reducido $\phi'(Y \setminus \{E\}) = \phi[Y \setminus \{E\}]_{E=e}$ se define para cualquier asignación y compatible con e como:

$$\phi'(y) = \phi(y, E = e)$$

Operación de Multiplicación (Producto de Factores)

Sean $\phi_1(X)$ y $\phi_2(Y)$ dos factores. El producto $\psi(X \cup Y) = \phi_1 \times \phi_2$ se define como:

$$\psi(x \cup y) = \phi_1(x) \cdot \phi_2(y)$$

donde las asignaciones x y y coinciden en las variables compartidas $X \cap Y$.

Operación de Marginación (Suma hacia fuera)

Sea $\phi(Y)$ un factor y $Z \in Y$. La marginación de Z, denotada como $\tau(Y \setminus \{Z\}) = \sum_Z \phi\,(Y)$, se define como:

$$\tau(y \setminus \{z\}) = \sum_{z \in \mathrm{Val}(Z)} \phi\,(y, \setminus \{z\} Z = z)$$

Algoritmo de Eliminación de Variables (Variable Elimination)

Dada una variable de consulta Q, un conjunto de evidencia observada $E = e$, y un orden de eliminación para las variables irrelevantes $Z = \mathcal{V} \setminus (\{Q\} \cup E)$:

$$\pi = (Z_1, Z_2, \ldots, Z_m)$$

El algoritmo procede computando la distribución condicional $P(Q \mid E = e)$ sin reconstruir la tabla conjunta global:

1. **Inicialización:** Crear el conjunto inicial de factores $\Phi = \{\phi_{X_i} \equiv P(X_i \mid \mathrm{Padres}(X_i))\}_{i=1}^{n}$.
2. **Incorporación de Evidencia:** Para cada factor $\phi \in \Phi$, reducirlo fijando las variables observadas en $E = e$.
3. **Bucle de Eliminación:** Para cada variable $Z_j \in \pi$:
    - Identificar los factores en Φ cuyo alcance contenga a Z_j:

$$\Phi_{Z_j} = \{\phi \in \Phi \mid Z_j \in \text{Scope}(\phi)\}$$

- Calcular el nuevo factor resultante mediante el producto y posterior suma de Z_j:

$$\psi_{\text{nuevo}} = \sum_{Z_j} \left(\prod_{\phi \in \Phi_{Z_j}} \phi \right)$$

- Actualizar el conjunto de factores:

$$\Phi \leftarrow \left(\Phi \setminus \Phi_{Z_j}\right) \cup \{\psi_{\text{nuevo}}\}$$

4. **Producto Final y Normalización:** Multiplicar todos los factores remanentes (cuyo alcance es únicamente el conjunto $\{Q\}$) y normalizar sobre los estados de Q:

$$\alpha = \sum_{q \in \text{Val}(Q)} \phi_{\text{final}}\,(Q = q)$$

$$P(Q = q \mid E = e) = \frac{\phi_{\text{final}}(Q = q)}{\alpha}$$

JUSTIFICACIÓN DE LAS HERRAMIENTAS SELECCIONADAS

Capa del Sistema	Herramienta / Tecnología	Versión	Justificación Técnica
Lenguaje Base	Python	3.11+	Sintaxis expresiva, ecosistema científico robusto, tipado estático opcional (typing) y alta portabilidad nativa.
Framework GUI	PyQt6 / PySide6 (Qt)	6.6+	Basado en el estándar industrial Qt. Provee el componente QGraphicsView / QGraphicsScene idóneo para renderizar grafos interactivos a 60 FPS con aceleración 2D, además de un motor robusto de eventos.
Cálculo Numérico	NumPy	1.26+	Manejo de arreglos multidimensionales en memoria contigua en lenguaje C. Permite indexación booleana eficiente, productos tensoriales y sumas en ejes específicos sin sobrecarga del bucle de Python.
Estructuras de Grafos	NetworkX	3.2+	Biblioteca estándar para el análisis de redes complejas. Utilizada internamente para la validación topológica instantánea (detección de ciclos con algoritmos de Tarjan/DFS) y cálculo de órdenes de eliminación.

Capa del Sistema	Herramienta / Tecnología	Versión	Justificación Técnica
Serialización	JSON estándar	RFC 8259	Estructura legible, ligera y portable. Permite guardar la topología, posiciones en el lienzo X, Y, metadatos y matrices de CPT en un solo archivo plano.
Pruebas y Calidad	PyTest + Black / Flake8	Última	Suite moderna para la ejecución automatizada de pruebas unitarias y de integración, garantizando reproducibilidad y código limpio conforme a PEP 8.

ARQUITECTURA DEL SOFTWARE Y DISEÑO DE COMPONENTES

El sistema se estructura bajo el patrón arquitectónico **Modelo-Vista-Controlador (MVC)** desacoplado, complementado con un subsistema de mensajería reactiva a través del sistema de señales y ranuras (*Signals & Slots*) de Qt

Desglose de Componentes

1. **GraphCanvasView (Vista):** Hereda de QGraphicsView. Captura eventos de mouse (clic para crear nodo, arrastrar para crear conexión, selecciones múltiples) y dibuja los elementos visuales.

2. **NodeItem / EdgeItem (Vista):** Representaciones vectoriales de nodos y aristas dirigidas (flechas bezier/lineales) en el lienzo de QGraphicsScene.
3. **CPTEditorDialog (Vista):** Diálogo interactivo que autogenera dinámicamente columnas y filas según la combinación cartesiana de estados de los padres del nodo seleccionado.
4. **BayesianNetwork (Modelo):** Administra el grafo global, mantiene la consistencia de padres e hijos, valida que el grafo sea un DAG y almacena los objetos CPD.
5. **Factor (Modelo Matemático):** Encapsula un tensor numpy.ndarray junto con la lista de nombres de variables asociadas y sus cardinalidades, proveyendo métodos atómicos: .product(), .marginalize(), .reduce().
6. **VariableEliminationEngine (Servicio de Inferencia):** Implementa la orquestación del algoritmo, cálculo de heurísticas de ordenamiento y generación del registro de ejecución (*logging steps*).

EPRESENTACIÓN INTERNA DE NODOS, RELACIONES Y DISTRIBUCIONES

Estructura del Nodo (Node)

Cada nodo contiene metadatos gráficos y su definición aleatoria discreta:

- id (str): Identificador unívoco alfanumérico (ej. "Lluvia").
- name (str): Etiqueta visual para la interfaz.
- states (list[str]): Lista exhaustiva y mutuamente excluyente de categorías (ej. ["Verdadero", "Falso"] o ["Bajo", "Medio", "Alto"]).
- position (tuple[float, float]): Coordenadas cartesianas (x, y) en el lienzo gráfico.

Estructura de Relaciones (Aristas)

Las dependencias se gestionan internamente en la clase BayesianNetwork mediante listas de adyacencia directa e inversa para consultas en $\mathcal{O}(1)$:

- _parents: dict[str, list[str]]
- _children: dict[str, list[str]]

Representación de la CPT / CPD

Una distribución de probabilidad condicional $P(X, |, U_1, U_2 \dots U_k)$ se modela como un tensor de NumPy multidimensional cuyas dimensiones coinciden con la cardinalidad del nodo y de sus respectivos padres:

$$\text{Forma del Tensor} = (|, \text{Val}, (X), || \ \text{Val}(U_1) \ | \cdots | \ \text{Val}(U_k) \ |)$$

Condición estricta de validación: Para cualquier combinación fija de valores de los padres $u_1, \dots, u_k$, la suma a lo largo del eje correspondiente a la variable dependiente X debe satisfacer:

$$\sum_{x \in \mathrm{Val}(X)} P\,(X, =, x \mid u_1 \ldots u_k) = 1.0 \pm 10^{-5}$$

EXPLICACIÓN DETALLADA DEL ALGORITMO IMPLEMENTADO

El motor central del sistema se fundamenta en la manipulación declarativa de objetos Factor. A continuación se desglosan las rutinas nucleares:

Multiplicación de Factores (factor_product)

Para multiplicar $\phi_1(X)$ y $\phi_2(Y)$, se construye un alcance unificado $W = X \cup Y$.

1. Se expanden las dimensiones de cada tensor en NumPy utilizando np.reshape y np.broadcast_to para alinearlas con la unión de variables W.
2. Se ejecuta la multiplicación elemento a elemento sobre los arreglos transmitidos:

```
def multiply_factors(f1: Factor, f2: Factor) -> Factor:
    union_vars = list(dict.fromkeys(f1.variables + f2.variables))

    # Obtener ejes para el reshape
    shape_f1 = [f1.cardinality[var] if var in f1.variables else 1 for
var in union_vars]
    shape_f2 = [f2.cardinality[var] if var in f2.variables else 1 for
var in union_vars]

    data1 = f1.values.reshape(shape_f1)
    data2 = f2.values.reshape(shape_f2)

    # Multiplicación tensorial con multidimensional broadcasting
    new_values = data1 * data2
    new_cardinality = {v: (f1.cardinality.get(v) or f2.cardinality[v])
for v in union_vars}

    return Factor(variables=union_vars, cardinality=new_cardinality,
values=new_values)
```

Marginación de Variables (factor_marginalize)

Deseamos remover la variable Z del factor $\phi(Y)$. Se localiza el eje correspondiente a Z dentro del tensor y se ejecuta la suma vectorial a lo largo de dicho eje:

```
def marginalize_factor(f: Factor, variable: str) -> Factor:
    if variable not in f.variables:
        return f

    axis = f.variables.index(variable)
    new_values = np.sum(f.values, axis=axis)
    new_vars = [v for v in f.variables if v != variable]
    new_card = {v: f.cardinality[v] for v in new_vars}

    return Factor(variables=new_vars, cardinality=new_card,
values=new_values)
```

Heurísticas de Orden de Eliminación

El orden en que se eliminan las variables intermedias define el tamaño del factor más grande generado durante la ejecución (ancho del árbol / *treewidth*), condicionando el uso de memoria de la inferencia. Se incorporaron dos heurísticas codiciosas (*greedy*):

1. **Min-Neighbors:** En cada paso, selecciona la variable cuyo nodo en el grafo moralizado tenga la menor cantidad de vecinos adyacentes.
2. **Min-Weight:** Selecciona la variable que minimice el producto de las cardinalidades de los factores vecinos a fusionarse:

$$\arg\min_{Z_j} \prod_{V \in \bigcup \text{Scope}(\phi), \phi \in \Phi_{Z_j}} |\text{Val}(V)|$$

DESCRIPCIÓN DE LA INTERFAZ Y FUNCIONALIDADES

La interfaz gráfica fue construida con una estética profesional, soporte de tema oscuro (*Dark Mode*) y paneles modulares acoplables (QDockWidget).

Módulos y Funcionalidades Principales

1. **Barra de Herramientas del Grafo:**
    - Modo Selección: Arrastre libre de nodos con reubicación automática de las líneas de los arcos conectores.
    - Modo Creación de Nodos: Inserción rápida con generación automática de identificadores sin colisión.
    - Modo Conexión: Creación de aristas mediante arrastrar y soltar entre terminales de conexión de nodos.
2. **Editor Avanzado de CPT:**
    - Despliegue en matriz tabular donde los encabezados se reconfiguran dinámicamente si el nodo adquiere o pierde padres en el lienzo.
    - Validación visual inmediata: las celdas cambian a color rojo si la suma no da 1.0, o a verde cuando el vector estocástico es válido.
    - Botón de auto-distribución uniforme (normalización automática rápida).
3. **Panel de Inferencia y Evidencia:**
    - Menú desplegable para seleccionar la variable objetivo Q.

- Lista dinámica de casillas de verificación para marcar cualquier subconjunto de nodos como evidencia E y asignar sus valores observados correspondientes.

4. **Visor de Traza de Factores (Paso a Paso):**
   - Registro interactivo que desglosa cada multiplicación y marginación con su respectiva fórmula LaTeX renderizada, permitiendo verificar con fines pedagógicos la secuencia exacta de operaciones algebraicas.
5. **Importación / Exportación:**
   - Serialización de archivos .json que guardan la red completa, listas de estados, matrices de CPT y la posición estética en píxeles de cada nodo.

PRUEBAS, RESULTADOS, LIMITACIONES Y POSIBLES MEJORAS

Caso de Validación 1: Red del Alarma de Pearl (Earthquake / Burglary)

Se modeló la arquitectura clásica de Judea Pearl:

- Variables: Robo (B), Terremoto (E), Alarma (A), JohnCalls (J), MaryCalls (M).
- Consulta ejecutada: $P(B, = \text{True} \mid J = \text{True} M = \text{True})$.
- **Resultado en nuestra herramienta:** $P(B = \text{True}) \approx 0.28417$.
- **Validación cruzada contra pgmpy:** 0.2841715 (coincidencia con error relativo $< 10^{-6}$).

Caso de Validación 2: Red Estudiante (Student Network - Koller & Friedman)

- Variables: Dificultad (D), Inteligencia (I), Calificación (G), SAT (S), Carta (L).
- Consulta ejecutada: $P\left(I, = \text{Alta} \mid G = \text{Grado } 3 D = \text{Fácil}\right)$.
- **Resultado obtenido:** 0.0788, validando de forma exacta la propagación bidireccional de evidencias y el fenómeno de inter-explicación (*explaining away*).

Matriz de Pruebas Unitarias

El proyecto cuenta con una batería de pruebas automatizadas en pytest:

- test_acyclic_enforcement: Intento de añadir aristas cíclicas ($A \rightarrow B \rightarrow C \rightarrow A$) arroja la excepción CycleDetectedException.
- test_factor_marginalization: Verificación del colapso del tensor contra sumas vectoriales canónicas.
- test_unnormalized_cpt: El sistema bloquea la ejecución de inferencias si existe al menos una columna con suma diferente de 1.0.

Limitaciones del Sistema

1. **Variables Continuas:** El software solo admite variables aleatorias categóricas discretas; no soporta distribuciones continuas (Gaussianas / CLG).
2. **Explosión Combinatoria:** Si el grafo tiene un ancho de árbol muy grande y muchas variables interconectadas sin independencias condicionales aprovechables, el factor intermedio supera la memoria RAM disponible ($\mathcal{O}(k^w)$).
3. **Inferencia Aproximada Ausente:** No cuenta con algoritmos de muestreo estocástico (MCMC, Gibbs Sampling, Likelihood Weighting) para redes de gran escala donde la inferencia exacta no es viable.

Posibles Mejoras Futuras

- Implementación de algoritmos de paso de mensajes en árboles de uniones (*Junction Tree / Clique Tree Calibration*) para responder múltiples consultas simultáneas en una sola pasada.
- Soporte para variables continuas mediante discretización adaptativa o distribuciones paramétricas mixtas.
- Exportación e importación en formato XMLBIF estándar para interoperabilidad directa con GeNIe y Weka.

REPOSITORIO OFICIAL DE GITHUB

El código fuente, manual de instalación, especificación de requerimientos (requirements.txt), archivos de prueba y diagramas de arquitectura se encuentran alojados públicamente en:

- **Enlace al Repositorio**: https://moisesmorenoumsnh.github.io/Proyecto-Integrador/

CONCLUSIONES

1. Se implementó exitosamente una herramienta integral que une el rigor formal de los Modelos Gráficos Probabilistas con un entorno visual dinámico, intuitivo y pedagógico.
2. El algoritmo de Eliminación de Variables demostró ser altamente eficiente en comparación con la enumeración completa, reduciendo los tiempos de respuesta de segundos a milisegundos en redes de tamaño moderado mediante la aplicación de las heurísticas de ordenamiento Min-Weight.
3. La modularidad logrado a través de la arquitectura MVC facilitó el desarrollo desacoplado, permitiendo validar las operaciones matemáticas sobre factores con pruebas unitarias independientes de la interfaz gráfica en PyQt6.
4. El desarrollo del proyecto consolidó el entendimiento práctico de las propiedades de independencia condicional, la factorización de distribuciones conjuntas y la complejidad algorítmica inherente al razonamiento automatizado bajo incertidumbre.

DISTRIBUCIÓN DE RESPONSABILIDADES Y CONTRIBUCIONES INDIVIDUALES

Este software fue desarrollado como proyecto integrador para la materia de **Modelos Probabilistas**, correspondiente al programa de Ingeniería en Computación de la **Facultad de Ingeniería Eléctrica (FIE)** en la **Universidad Michoacana de San Nicolás de Hidalgo (UMSNH)**.

Integrantes y División de Responsabilidades

Integrante	**Rol**	**Responsabilidades y Contribuciones Técnicas**
Moisés P. C.	**Líder Técnico & Arquitectura**	Arquitectura desacoplada del sistema, motor matemático de factores multidimensionales, implementación del algoritmo de Eliminación de Variables (6 pasos), motor de enumeración exhaustiva y gestión del estado global con Zustand (useStoreRed.ts).
César Ambris Salguero	**Desarrollo Frontend & Reportes**	Diseño y maquetación de paneles visuales (BarraSuperior.tsx, PanelEditor.tsx), maquetación de vista imprimible y exportación a PDF/Markdown (VistaReporte.tsx), y diseño de modelos probabilistas de 5 y 6 nodos (Videojuegos y Ransomware).

Integrante	Rol	Responsabilidades y Contribuciones Técnicas
Leonardo Soria Raya	**Persistencia, Validadores & Modelado**	Esquemas de serialización JSON con Zod, validadores de probabilidad total y aciclicidad, utilidades del sistema (descargarArchivo.ts), y diseño de modelos probabilistas de 4, 7 (multivaluado) y 10 nodos (Mundial de Fútbol).